PLATAFORMA TRANSACCIONAL PARA EVENTOS DE ALTA DEMANDA

Sistema Distribuido de Venta de Entradas

Cero Sobreventas, Escalabilidad en la Nube y Seguridad de Nivel Bancario

Solución integral basada en microservicios independientes que garantiza ventas exactas en milisegundos, protección total ante picos masivos de tráfico y continuidad operativa sin caídas del sistema.

Identidad y Accesos

User Service (:3000) - Marcos

- Onboarding rápido de clientes
- Encriptación Bcrypt de claves
- Sesiones rápidas con JWT

Motor de Stock Exacto

Inventory Service (:3001) - Nahuel

- Catálogo en tiempo real
- Bloqueo en memoria con Redis
- Garantía de cero sobreventas

Checkout Resiliente

Checkout Service (:3002) - Lisandro

- Procesamiento ágil de compras
- Tolerancia a fallos de red
- Escalabilidad horizontal

Tecnología: Node.js 20 | Express | MongoDB 7 | Redis 7 | JWT | Docker Compose | OpenAPI Swagger

1. El Desafío del Mercado y Nuestra Propuesta de Valor

PROPUESTA DE VALOR

¿Por qué fallan las ticketeras tradicionales en eventos masivos?

- Cuando miles de fanáticos intentan comprar las últimas entradas al mismo segundo, los sistemas comunes colapsan o venden dos veces la misma ubicación (sobreventa), generando reclamos legales y pérdida de reputación.
- Nuestra plataforma resuelve este problema de raíz combinando bloqueo ultrarrápido en memoria y microservicios desacoplados.

1. Cero Sobreventas

- Bloqueo instantáneo por producto en Redis.
- Es físicamente imposible vender más entradas de las disponibles.
- Respuesta en milisegundos sin saturar la base de datos.

2. Siempre Disponible

- Módulos independientes: el catálogo y las compras no dependen del servidor de registro.
- Si un servicio se demora, el sistema responde en menos de 3 segundos sin colgarse.

3. Listo para Escalar

- Empaquetado en contenedores Docker listos para la nube.
- Permite multiplicar el servicio de compras en horas pico (Hot Sale) sin frenar el negocio.

2. Arquitectura de Microservicios Independientes

ARQUITECTURA

Ecosistema Conectado en Red Privada de Alta Velocidad (backend-net)

user-service (:3000)

Clientes, Login y Tokens JWT

BD Exclusiva: users-db

checkout-service (:3002)

Orquestador Ágil de Compras

100% Stateless - Escala al instante

PUT /reducir

---------->

inventory-service (:3001)

Custodio del Stock en Vivo

Redis (Lock) + MongoDB (Stock)

Aislamiento Total de Datos por Dominio

- Cada área del negocio protege su propia base de datos sin interferencias externas.
- Inventory Service encapsula tanto el stock en MongoDB como el motor de bloqueos en Redis.
- Checkout procesa órdenes sin guardar estado local, permitiendo sumar servidores en segundos.

Beneficios Directos para el Negocio

- Continuidad de ventas: Los usuarios logueados siguen comprando incluso si el módulo de registro entra en mantenimiento.
- Mantenimiento sin cortes: Se puede actualizar un servicio sin apagar el resto de la plataforma.
- Menor costo operativo: Se escalan solo los módulos con mayor demanda.

3. Motor Anti-Sobreventa en Tiempo Real y Resiliencia

TECNOLOGÍA CLAVE

Doble Barrera contra la Sobreventa

1. Escudo en Memoria RAM (Redis): Ante dos compras al mismo milisegundo, la primera toma un cerrojo exclusivo por 5 segundos (SET NX PX 5000).

2. Rechazo Inmediato de Colisiones: La segunda recibe aviso instantáneo (409 Conflict) sin sobrecargar la base.

3. Descuento Atómico en MongoDB: Verifica y descuenta en un único paso indivisible (stock > 0).

4. Desbloqueo Seguro: Un Script Lua libera el producto al instante para el siguiente comprador.

Tolerancia a Fallos y Continuidad Operativa

- Corte Inteligente a los 3 Segundos: Si la red interna sufre una demora, Checkout cancela la espera automáticamente (AbortController).

- Cero Efecto Dominó: Ningún fallo externo puede colgar ni reiniciar el servidor de ventas.

- Recuperación Automática: Cuando el servicio vuelve a estar en línea, las ventas se reanudan solas sin intervención técnica.

Corazón del Motor Transaccional en Inventory Service

```
lock = await redisClient.set(lockKey, ownerId, { NX: true, PX: 5000 }); // Bloqueo en < 1ms
```

```
await Inventario.findOneAndUpdate({ producto, stock: { $gt: 0 } }, { $inc: { stock: -1 } });
```

```
// Liberación atómica en Redis mediante Script Lua verificando la identidad de la operación
```

4. Seguridad Corporativa, Auditoría y Despliegue Cloud

SEGURIDAD & CLOUD

Seguridad Multicapa de Extremo a Extremo

- Protección de Credenciales: Contraseñas encriptadas con estándar industrial Bcrypt (10 rondas de sal).

- Tokens Criptográficos JWT (HS256): Identidad verificada en cada microservicio sin viajes extras por la red.

- Permisos por Rol (RBAC):

  * Clientes (USER): Compra de entradas y acceso exclusivo a su propio perfil.

  * Administradores (ADMIN): Gestión de catálogo, reposición de stock y auditoría de usuarios.

- Bases de Datos Blindadas: MongoDB y Redis operan ocultos dentro de la red interna sin exposición a Internet.

Infraestructura Lista para Producción

- Despliegue Instantáneo con Docker Compose: Los 6 contenedores (3 servicios + 2 MongoDB + 1 Redis) levantan con un solo comando.

- Autodiagnóstico (Healthchecks): Monitoreo continuo del estado de cada servicio y base de datos.

- Trazabilidad Total (Winston): Registro estructurado de cada compra, intento fallido o alerta de seguridad.

- Integración Inmediata (Swagger OpenAPI): Portales interactivos listos para conectar con cualquier App móvil o Web.

```
docker compose up -d --build
```

5. Calidad Comprobada en Vivo y Resumen Ejecutivo

DEMO & CIERRE

Batería de 22 Pruebas Automatizadas de Estrés y Seguridad (npm test)

1. Verificación de Infraestructura: Comprueba en vivo la salud de los 3 microservicios, MongoDB y Redis.

2. Simulación Real de Alta Concurrencia: Dispara compras simultáneas al mismo milisegundo por la última entrada disponible y demuestra que solo 1 se concreta y el stock jamás queda negativo.

3. Blindaje de Seguridad: Verifica automáticamente que usuarios sin permiso o con tokens inválidos sean bloqueados en el acto (códigos 401 y 403).

Demostración en Vivo en 2 Comandos

1. Iniciar toda la plataforma:

```
docker compose up -d --build
```

2. Ejecutar auditoría automática (22/22 OK):

```
npm test
```

¿Por qué elegir nuestra plataforma?

- Exactitud garantizada: Cero sobreventas.
- Alta disponibilidad: Sin puntos únicos de caída.
- Seguridad probada: JWT, Bcrypt y red aislada.
- Lista para crecer: Escalable en segundos con Docker.